

Backend — fiche de révision

Tout ce que le jury peut demander sur l'API, avec les réponses. Établi par lecture du code, pas de mémoire : chaque chiffre a été compté, chaque affirmation vérifiée.

Ce qui est faux ou fragile est signalé ⚠️. Mieux vaut le découvrir ici que devant eux.

Sommaire

- 1. Les chiffres à connaître par cœur
- 2. Le chemin d'une requête
- 3. Le socle
- 4. La couche de données
- 5. La sécurité
- 6. Le métier
- 7. Les tests et l'intégration continue
- 8. Le déploiement
- 9. 25 questions probables
- 10. Les faiblesses — à assumer, à corriger

1. Les chiffres à connaître par cœur

Modules de routes	25
Points d'entrée	133 — 70 GET, 26 POST, 21 PUT, 13 DELETE, 3 PATCH
Contrôleurs	27
Services · utilitaires · middlewares · tâches	4 · 7 · 3 · 2
Lignes dans <code>src/</code>	9 967
Modèles Prisma	36, dont 16 énumérations
Relations	48 — dont 37 en cascade, 8 en <code>SetNull</code>
Index explicites · contraintes d'unicité composites	13 · 9
Migrations	5
Fichiers de tests · cas écrits · cas réellement exécutés	13 · 168 · 126 ⚠️

Pile : Node 22, Express 5, Prisma 5.9, PostgreSQL (Supabase), Redis facultatif, bcrypt, jsonwebtoken, helmet, express-rate-limit, multer, node-cron, rrule, ws.

2. Le chemin d'une requête

C'est la réponse à « expliquez-moi ce qui se passe quand j'appelle votre API ». Sache la dérouler dans l'ordre — l'ordre **est** la réponse.

Requête HTTPS

```
|
| 1. trust proxy      (production) sans lui, req.ip = IP du proxy Fly
| 2. helmet           en-têtes de sécurité – avant tout, pour couvrir même les 404
| 3. compression      gzip – avant les routes, pour couvrir toutes les réponses
| 4. cors              origines listées en production, jamais « * »
| 5. express.json      limite 10 Mo → peuple req.body
| 6. /uploads          fichiers statiques – AVANT le limiteur, donc hors quota
| 7. apiLimiter         300 req/min/IP sur /api
| 8. le routeur        authenticate → authorize(rôle) → contrôleur
|    └─ contrôleur : vérification de PROPRIÉTÉ, puis Prisma, puis sendSuccess
| 9. catch-all 404     après toutes les routes, sinon il les absorberait
|    └─ 10. gestionnaire d'erreurs (err, req, res, next) – dernier, Express le reconnaît
|                                     à son arité de 4 arguments
```

Les deux inversions à ne pas faire, et savoir pourquoi : le 404 avant les routes les avalerait toutes ; le gestionnaire d'erreurs avant le 404 deviendrait inatteignable.

3. Le socle

3.1 La configuration refuse de démarrer sans secret

src/config/env.js — fonction `required(nom, repliDev)` :

- variable présente → renvoyée ;
- absente **et** `NODE_ENV=production` → **exception au chargement du module**, donc avant que le serveur n'écoute. Message : « *Le démarrage est interrompu volontairement — un repli en production exposerait l'application* » ;
- absente hors production → avertissement console et repli de développement.

Deux variables sont obligatoires en production : `DATABASE_URL` et `JWT_SECRET` . Le commentaire du code dit l'essentiel : « *un secret connu permet de forger n'importe quel jeton* ».

La phrase à dire : « *Un démarrage qui échoue est préférable à un démarrage silencieux avec un secret par défaut.* »

3.2 Le format de réponse uniforme

src/utils/responseHandler.js , 29 lignes, deux fonctions :

```
sendSuccess(res, data, message, statusCode = 200) → { success: true, message, data }
sendError(res, message, statusCode = 500, errors) → { success: false, message, errors }
```

Même les réponses qui ne passent pas par ces fonctions respectent le contrat : le 404 attrape-tout, le 429 du limiteur, les erreurs multer, les 403 de modération.

Pourquoi ça compte — et c'est l'argument fort : côté client, un **seul** intercepteur d'erreur suffit, au lieu d'un par appel. Une convention tenue vaut mieux qu'une abstraction élégante.

3.3 Le client Prisma est un singleton

`src/config/database.js` : une seule instance, exportée par défaut. Le cache de modules ES garantit un seul objet pour tout le processus.

Pourquoi ? Chaque `new PrismaClient()` ouvre son propre pool de connexions. En multiplier revient à épuiser le `max_connections` de PostgreSQL.

Journalisation : `['error', 'warn']` en développement, `['error']` sinon. **Les requêtes ne sont jamais journalisées** — pour ne pas déverser de données personnelles dans les logs.

Double URL de connexion (`schema.prisma`) : `url` passe par le pooler pgbouncer (port 6543, mode transaction), `directUrl` par le port 5432. Les migrations exigent une session persistante que le pooler en mode transaction n'accorde pas.

⚠ Nuance à connaître : `database.js` reconstruit l'URL au démarrage, donc `directUrl` n'est utilisé que par la CLI Prisma, pas à l'exécution.

3.4 Redis : un cache, et un seul

Ce qu'il cache : le compteur de notifications non lues, appelé par chaque application ouverte **toutes les 30 secondes**. C'est l'endpoint le plus sollicité de l'API, pour une valeur qui bouge peu.

Clé	<code>notif:unread:<userId></code> — une par utilisateur, l'invalidation n'en touche jamais une autre
TTL	60 s — filet si une invalidation était oubliée quelque part
Écriture	invalidation par <code>DEL</code> , pas par recalcul : la prochaine lecture s'en charge
Reconnexion	3 tentatives, délai <code>essai × 200 ms</code> plafonné à 1 s

La dégradation propre est le point à mettre en avant : `client` vaut `null` tant qu'aucune connexion n'a abouti, et **tous** les appelants le testent. Si Redis tombe, l'application interroge PostgreSQL — elle ralentit, elle ne s'arrête pas.

Détail qui montre qu'on connaît la bibliothèque : l'écouteur `error` est **obligatoire**. Sans lui, une erreur de connexion devient une exception non capturée qui arrête le processus.

3.5 Le stockage de fichiers, appris en production

Multer garde le fichier **en mémoire**, pas sur disque : c'est le service de stockage qui décide de la destination. Écrire d'abord sur disque pour re-lire et ré-envoyer laisserait des fichiers orphelins si le dépôt distant échouait.

Deux pilotes derrière une seule fonction `saveUpload(file)`, choisis par `STORAGE_DRIVER` : disque local en développement, **Supabase Storage en production**.

Pourquoi ? Le système de fichiers d'une machine Fly est réinitialisé à chaque déploiement. « Une photo de profil déposée le lundi aurait disparu le mardi. »

Le nom du fichier est un secret : `randomUUID()` et non `Math.random()`. Le bucket est public et `/uploads` est servi sans authentification — une balise `<img>` ne peut pas porter d'en-tête `Authorization`. Le nom imprévisible est donc la seule protection de la ressource. C'est de la sécurité par URL non devinable, présentée comme telle.

4. La couche de données

4.1 Le modèle en sept domaines

Identité · Modération/RGPD · Entraînement · Nutrition et suivi · Communication et agenda · Géographie · Réputation.

La séparation structurante : ce qui est **prescrit** (programme → séance → exercice) et ce qui est **réalisé** (`SetCompletion`, `DailyStat`) sont deux grappes distinctes. C'est leur écart qui a de la valeur : sans cette séparation, impossible de dire si un client a suivi son programme.

4.2 La relation coach-client

Table de liaison explicite `ClientCoach`, porteuse d'attributs : `isPrimary`, `startDate`, `endDate`, `isActive`. Contrainte `@@unique([clientId, coachId])` — un couple n'existe qu'une fois, on réactive par `upsert` plutôt que de créer un doublon.

⚠ **Le champ hérité** `ClientProfile.coachId` **existe toujours**, commenté « rétrocompatibilité ». Il est encore écrit et encore lu à deux endroits. C'est une double source de vérité non résorbée — et c'est **exactement** ce qui a causé le bug de l'écran d'analyse. Si on te le demande, la réponse honnête : « la migration vers la table de liaison n'est pas terminée ; le champ devrait être supprimé, et deux requêtes réécrites. »

4.3 Cascades : la logique à énoncer

37 cascades, 8 `SetNull`. La règle est simple et se défend en une phrase :

Cascade sur les données dérivées — profils, programmes, séances, séries, repas, statistiques. **`SetNull` sur les données probantes** — journal de modération, historique de conversation, agenda.

Exemples de `SetNull` et leur raison : un signalement survit au départ du signalant (sinon supprimer son compte annulerait le signalement) ; une action de modération survit à l'administrateur qui l'a prise ; le message de proposition survit à l'annulation du rendez-vous.

Les cascades sont aussi ce qui rend le **droit à l'effacement** réellement applicable : supprimer un compte emporte ses données liées, sans script de nettoyage à maintenir.

4.4 Index et contraintes

13 index. Les mieux justifiés : `appointments(coachId, startAt)` pour l'agenda et la détection de conflit, `reports(status, createdAt)` et `appeals(status, createdAt)` qui correspondent **exactement** au `where` +

`orderBy` des listes de modération, `users(status, scheduledDeletionAt)` pour le balayage du cron de suppression.

⚠ Index manquants, à concéder si on creuse : `client_coaches(coachId, isActive)` — la requête la plus fréquente de l'application —, `messages(coachId, clientId, createdAt)`, et les clés étrangères de `programs`.

9 contraintes d'unicité composites, chacune protégeant une règle métier : `Session(programId, date)` — une séance par jour et par programme ; `SetCompletion(exerciseId, setNumber)` — une série numérotée enregistrée une fois ; `DailyStat(clientId, date)` ; `Review(coachId, clientId)` — un seul avis.

L'argument transversal : ces contraintes déplacent l'unicité **dans la base** plutôt que dans un `findFirst` suivi d'un `create`. C'est la seule protection fiable contre les doubles soumissions concurrentes.

4.5 Transactions

12 sites, dans 6 contrôleurs. Le fil directeur :

Transaction dès qu'un état partiel serait irrattrapable depuis l'interface.

Exemples : une sanction sans sa trace d'audit ; un rendez-vous sans le message qui porte les boutons Accepter/Refuser — « *une proposition orpheline, existante en base, invisible dans le fil* » ; le remplacement atomique d'un jeu de créneaux (`deleteMany` + `createMany`).

⚠ Là où elle manque : l'acceptation d'une demande lit les coachs existants **avant** d'ouvrir la transaction, pour calculer `isPrimary`. Deux acceptations concurrentes pourraient produire deux coachs principaux.

4.6 SQL brut

Trois emplacements, tous en `$queryRaw` avec *tagged template* — donc paramétrés. **Zéro** `$queryRawUnsafe`, zéro concaténation. C'est l'argument décisif sur l'injection.

Pourquoi du SQL brut malgré tout : la recherche d'aliments trie par pertinence (`ORDER BY CASE WHEN name ILIKE 'terme%' THEN 0 ELSE 1 END`), et la recherche de salles calcule une distance de Haversine — deux choses que l'API de requête de Prisma ne sait pas exprimer. Le calcul de distance est précédé d'un pré-filtre par boîte englobante, sans quoi on calculerait la distance sur plus de 6 000 lignes.

4.7 Migrations

Cinq migrations. En développement `prisma migrate dev`, en CI `prisma db push` sur une base jetable, en production `prisma migrate deploy`, **appliqué à la main**.

⚠ Deux aveux à préparer : il n'y a **pas de** `release_command` dans `fly.toml`, donc la migration n'est pas automatisée ; et la CI utilisant `db push`, **elle ne teste jamais l'exécutabilité des migrations**.

La règle que tu appliques : les migrations d'abord, le code ensuite. Déployer du code qui attend une colonne absente ne casse pas une page, ça casse toutes les connexions.

4.8 Pagination

⚠ **Une seule route est réellement paginée** : la recherche d'aliments (`page` , `pageSize=20` , avec le total). Ailleurs, deux régimes : un plafond `take` fixe sans `skip` (modération 100, notifications 50, salles 30), ou **rien du tout** — conversation complète, programmes d'un coach avec toutes leurs séances et exercices, liste de clients.

Le bon exemple à citer : la recherche de salles fait `take: limit + 1` et renvoie un drapeau `truncated` . **C'est le seul endroit qui signale la troncature au lieu de la masquer.**

5. La sécurité

5.1 Mots de passe

bcrypt, **coût 10** ($2^{10} = 1\,024$ itérations), sel intégré et généré par la bibliothèque. Deux comptes avec le même mot de passe ont des empreintes différentes — les tables pré-calculées sont inopérantes.

Le hash n'est jamais renvoyé : à l'inscription par une **projection** `select` **explicite**, à la connexion par déstructuration avant réponse, et il est **exclu de l'export RGPD** — exporter une empreinte reviendrait à distribuer une cible.

Message d'erreur uniforme à la connexion : « Email ou mot de passe incorrect » dans les deux cas → pas d'énumération de comptes. Subtilité à valoriser : le contrôle du statut de modération vient **après** la vérification du mot de passe — annoncer « compte suspendu » à un inconnu révélerait l'existence du compte.

⚠ **Le mot de passe exige 6 caractères, sans aucune contrainte de complexité.** C'est le point faible le plus assumable : l'ANSSI recommande 12 caractères sans autre contrainte. Dis-le avant qu'on te le demande.

5.2 Le jeton JWT

Bibliothèque	<code>jsonwebtoken</code>
Algorithme	HS256 (défaut, non explicite ⚠)
Durée	30 jours
Charge utile	<code>{ userId, role }</code> + <code>iat</code> , <code>exp</code> . Rien d'autre.

Le point le plus intéressant à faire valoir : le rôle porté par le jeton **n'est jamais utilisé pour autoriser**. À chaque requête, le middleware recharge l'utilisateur en base et c'est le rôle **et le statut en base** qui font foi.

Conséquence : un jeton émis avant une suspension ou un changement de rôle ne donne aucun privilège périmé. Coût assumé : une requête SQL par appel authentifié.

⚠ **La limite, à annoncer soi-même** : 30 jours, **aucune révocation**, pas de jeton de rafraîchissement. La seule révocation possible est la rotation du `JWT_SECRET`, qui déconnecte tout le monde. Le correctif est identifié — jeton court plus jeton de rafraîchissement, ou liste de révocation — il n'est pas implémenté.

5.3 L'autorisation à deux niveaux

C'est le morceau de bravoure de la partie sécurité. Prends ton temps dessus.

Niveau 1 — le rôle, sur la route. `authorize('COACH')` ne regarde que `req.user.role`. Il répond à « quel type d'acteur », jamais à « sur quel objet ». Deux styles de déclaration : routeur entier pour l'administration — « aucune route ne peut être ajoutée plus bas en oubliant sa protection » — ou route par route ailleurs.

Niveau 2 — la propriété, dans le contrôleur. Centralisée dans `utils/authorization.js`, 8 fonctions d'aide. Convention : chaque fonction renvoie `true / false`, le contrôleur choisit le code HTTP.

```
// Modification d'un programme – programController.js
if (!coachProfile || existingProgram.coachId !== coachProfile.id) {
  return sendError(res, "Vous n'êtes pas autorisé à modifier ce programme", 403);
}
```

La phrase clé : « `authorize('COACH')` dit tu es coach. Cette ligne dit tu es ce coach. Deux coaches ont le même rôle mais pas les mêmes clients. »

Sans le second niveau, n'importe quel coach lirait le programme d'un confrère en devinant un identifiant : c'est **A01 de l'OWASP Top 10 2021**, le contrôle d'accès défaillant au niveau de l'objet.

Les chaînes de propriété remontent le graphe : `canAccessExercise` → `Exercise` → `Session` → `Program`. `canAccessConversation` exige d'être l'un des deux participants — aucun rôle ne suffit.

⚠ **Le trou, et il est réel** : `sendMessage` ne vérifie **aucune propriété**. Toutes les lectures de conversation sont protégées, l'écriture ne l'est pas. Le champ `isSentByCoach` est même repris du corps de la requête. Voir partie 10.

5.4 Modération et RGPD

Trois statuts de compte : `ACTIVE`, `SUSPENDED`, `PENDING_DELETION`.

Le droit de recours prime sur la sanction. Deux middlewares d'authentification coexistent pour cette seule raison : une personne suspendue doit rester capable de contester, donc la route de recours a besoin de savoir **qui** appelle sans exiger un compte actif. `authenticateEvenIfSuspended` n'est utilisé nulle part ailleurs. À la connexion d'un compte suspendu, **un jeton est tout de même émis** — sans lui, pas de recours possible.

Transparence de la sanction : le 403 renvoie le motif, l'échéance et l'endpoint de recours. « Une sanction qu'on ne peut ni comprendre ni dater n'est pas contestable. »

Levée automatique à la volée : une suspension à terme échu est levée au premier accès, plutôt que par une tâche planifiée qui balaierait toute la table pour réactiver des comptes que personne n'utilise. Choix paresseux plutôt que par lot — défendable, et à savoir justifier.

Suppression différée : 30 jours, préavis à J-7, exécution par un cron à 3 h — **et un recours en cours suspend la suppression**. La trace `ModerationAction` est écrite **avant** la suppression : après, `targetId` serait nul et plus rien ne justifierait l'opération.

Minimisation : un administrateur ne voit **jamais** les données de santé, repas, programmes ou performances — une projection nommée le garantit. Une conversation signalée n'est lisible que tant que le signalement est ouvert. Toute consultation écrit un journal d'accès **avant** de répondre : si l'écriture du journal échoue, l'accès échoue.

Purge à 12 mois des signalements clos et des journaux d'accès — RGPD article 5.1.e, limitation de la conservation.

5.5 Défenses de surface

Mesure	Valeur
helmet	configuration par défaut, <code>crossOriginResourcePolicy</code> désactivé car <code>/uploads</code> sert des images au dashboard
CORS	liste d'origines en production, jamais <code>*</code> ; <code>*</code> seulement en développement
Limiteur d'authentification	10 tentatives ratées / 15 min / IP en production (100 hors production)
Limiteur général	300 requêtes / 60 s / IP
Corps de requête	10 Mo ⚠️ généreux
Fichier téléversé	5 Mo

Deux détails qui montrent l'expérience :

`skipSuccessfulRequests: true` sur l'authentification — **seuls les échecs comptent**, un utilisateur normal n'est jamais gêné.

Et le limiteur a été **déplacé** de `/api/auth` entier vers `/login` et `/register` seulement : `GET /me` répond 401 à chaque chargement de page sans session, et dix chargements suffisaient à bloquer la connexion pendant quinze minutes.

⚠️ Le compteur est en mémoire de processus, **non partagé entre les machines Fly**.

5.6 Injection, XSS, CSRF

Injection SQL — protégé structurellement. Prisma paramètre tout ; les trois requêtes brutes utilisent des *tagged templates* qui paramètrent aussi. Aucun `Unsafe` dans le projet.

XSS — non traité côté serveur, et c'est un choix à énoncer. L'API ne rend aucun HTML, elle ne renvoie que du JSON : pas de XSS réfléchi possible. Le **XSS stocké** est intégralement délégué au front. ⚠️ Si le dashboard faisait un `dangerouslySetInnerHTML` sur le contenu d'un message, la faille existerait et le backend l'aurait laissée passer. C'est une défense en profondeur manquante.

CSRF — pas de protection dédiée, et elle est inutile ici. L'authentification ne repose pas sur un cookie : le jeton voyage dans l'en-tête `Authorization`, qu'un navigateur n'ajoute **jamais** automatiquement sur une requête inter-site. Tout le principe du CSRF tombe. Le CORS restrictif est une seconde barrière.

Sache dire la réserve : `credentials: true` est activé en production alors qu'aucun cookie n'est utilisé. Inutile aujourd'hui, dangereux si un cookie de session était introduit plus tard.

5.7 Validation des entrées

Écrite **à la main**, en début de contrôleur, sans bibliothèque. Le bon exemple à montrer est la liste blanche :

```
const MOTIFS = ['HARASSMENT', 'SPAM', 'INAPPROPRIATE_CONTENT', 'FAKE_PROFILE', 'OTHER'];
if (!MOTIFS.includes(reason)) {
  return sendError(res, `Motif invalide. Valeurs acceptées : ${MOTIFS.join(', ')}`, 400);
}
```

Liste blanche, jamais liste noire, et un message qui indique les valeurs acceptées.

⚠ Ce qui manque, et il faut le dire : aucune borne supérieure de longueur nulle part ; `login` ne vérifie pas la présence de ses champs, donc un mot de passe absent produit un **500 au lieu d'un 400** ; pas de validation de format des identifiants ; pas de normalisation de l'email, donc `Marc@x.fr` et `marc@x.fr` créent deux comptes.

Le filet réel est **Prisma** : typage, énumérations, clés étrangères, unicité. Mais il produit un 500 là où un 400 serait juste.

Le correctif identifié : un schéma de validation déclaré par route, et un 400 systématique.

6. Le métier

6.1 Rendez-vous : la détection de conflit

Un seul `findFirst`, avec la condition de chevauchement classique :

```
existant.startAt < nouveau.endAt ET existant.endAt > nouveau.startAt
```

Le contact bord-à-bord n'est pas un conflit : un rendez-vous qui finit à 10 h 00 et un autre qui commence à 10 h 00 coexistent. C'est le comportement attendu, et les inégalités strictes le garantissent.

Filtre `status: 'CONFIRMED'` seulement. À la création on n'inspecte que l'agenda du coach ; à la confirmation, **coach ou client**, en s'excluant soi-même.

Trois résolutions proposées quand un conflit est détecté : le serveur renvoie un **409** avec le détail du rendez-vous en cause, et le client choisit — `replace` (supprime l'existant, et ses occurrences s'il s'agit d'une série), `shorten` (raccourcit celui qui commence en premier, avec un plancher d'une minute), ou abandon.

C'est le seul endroit de l'API où une erreur ouvre une **interface de décision** plutôt qu'un message.

6.2 Récurrence

Bibliothèque `rrule`, réservée aux coaches. Les occurrences sont **matérialisées en base** : un rendez-vous racine porte la chaîne `RRULE`, chaque occurrence est une ligne avec un `parentId`. Plafond volontaire à **un an**.

Les listes excluent les parents de série pour ne pas afficher deux fois la même chose. L'annulation et la suppression acceptent un `scope` — `single` par défaut, ou `series`.

Les trois écritures — racine, occurrences, message de proposition — sont dans **une seule transaction**.

6.3 Notifications et tâches planifiées

Tâche	Fréquence	Ce qu'elle fait
Rappel J-1	tous les jours à 8 h 00	message dans la conversation pour les rendez-vous des 24 h à venir
Rappel H-1	toutes les 5 minutes	notification au coach et au client, fenêtre <code>[+55 min, +65 min]</code>
Modération	tous les jours à 3 h 00	préavis à J-7, suppressions échues, purge à 12 mois

Deux détails de conception à savoir expliquer.

La fenêtre du rappel H-1 fait **10 minutes de large pour un passage toutes les 5** : délibérément plus large que l'intervalle, pour ne rien rater entre deux exécutions. Le doublon est évité par un verrou d'idempotence — le champ `reminderSentAt`, positionné après envoi et filtré à la sélection.

Le cron de modération tourne à **3 h du matin** : hors des heures d'usage, et la granularité du jour suffit pour des échéances exprimées en jours.

⚠ Limite architecturale honnête : avec plusieurs machines Fly, les crons s'exécuteraient sur chacune. Il n'y a **aucun verrou distribué** — l'idempotence est la seule protection, et le rappel J-1 n'en a pas.

6.4 Les services : pourquoi seulement quatre

Il n'y a **pas de couche service systématique** : la logique métier vit dans les 27 contrôleurs. Les quatre services existent chacun pour une raison précise, et c'est cette raison qu'il faut donner :

- `workoutData.js` — extraction faite parce que deux contrôleurs d'analyse partageaient la même requête et le même calcul de 1RM de référence. « *Sans cette extraction, la même logique aurait existé en deux exemplaires, avec la garantie de diverger.* »
- `notificationCache.js`, `fileStorage.js`, `overpass.js` — les trois points où le code parle à un **système extérieur** : Redis, Supabase Storage, OpenStreetMap.

La règle : on extrait quand la logique est dupliquée, ou quand elle touche l'extérieur. Pas par principe.

7. Les tests et l'intégration continue

7.1 Le dispositif

Runner `node:test` **natif** — pas de Jest, pas de Vitest, pas de Supertest. « *Pas de dépendance supplémentaire pour une fonction que la plateforme fournit.* » Les tests appellent l'API **en HTTP**, avec `fetch`, comme un vrai client.

Ce que `scripts/test-isolated.sh` met en place : une base `coaching_app_test` dédiée, un serveur en `NODE_ENV=test` sur le **port 5002**, attente active de `/api/health` (40 tentatives × 0,5 s), et un `trap EXIT` qui tue le serveur quoi qu'il arrive.

Le garde-fou anti-pollution est une bonne histoire à raconter : les tests interrogent `/api/health` et **refusent de démarrer si** `env !== 'test'`. Pourquoi ? Parce que plus de **160 comptes** `@test.com` avaient été semés dans la base de développement. C'est aussi la raison pour laquelle `/api/health` expose son environnement.

Exécution séquentielle (`--test-concurrency=1`) : les suites partagent une base. L'isolation entre cas repose sur l'unicité des données — un suffixe horodaté dans chaque email.

7.2 Le test de cloisonnement

Le meilleur test du projet. Il construit **deux écosystèmes complets et indépendants** — coach A avec ses clients, programmes, séances, repas, messages, templates ; coach B avec les siens — puis vérifie que B n'atteint **aucune** ressource de A.

Structure en **table de cas** : 18 tuples pour les accès interdits (attendu 403), 11 pour la non-régression (A garde accès à ses propres ressources, attendu 200). Protéger un nouvel endpoint revient à ajouter une ligne.

Les fixtures sont vérifiées avant les assertions : « *une fixture incomplète rendrait les assertions vides — on échoue explicitement* ».

***Pourquoi cet effort là** : ce sont des données de santé. Le cloisonnement est la règle métier la plus critique du produit ; elle mérite d'être testée, pas supposée. Et ce test est né d'une **faille réelle** : ces endpoints n'appliquaient aucune vérification d'appartenance.*

7.3 L'intégration continue

Un workflow, un job, déclenché sur `push` et `pull_request` vers `main` et `develop`. Service conteneurisé `postgres:15` avec healthcheck. Étapes : `checkout` → Node 22 (cache npm) → `npm ci` → recompilation de `bcrypt` → `prisma generate` → `prisma db push` → `npm test`.

Node 22 et non 20 : aligné sur la production, et le SDK Supabase exige un WebSocket natif absent de Node 20.

⚠ Ce qui manque : pas de lint, pas de build, pas d'audit de dépendances, pas de mesure de couverture, **pas de workflow de déploiement**, et aucune règle de protection de branche versionnée.

7.4 Ce qui n'est pas testé

À dire soi-même, dans cet ordre :

- ⚠ **Les rendez-vous : zéro test.** Ni la détection de conflit, ni les bornes de chevauchement, ni les résolutions, ni la génération RRULE, ni la règle « on ne confirme pas sa propre proposition ». **C'est la fonctionnalité la plus riche en règles métier et la moins couverte.**
- Les deux tâches planifiées : aucun test, alors que le module de modération exporte ses fonctions exprès pour être appelables.
- Les services : ni le repli quand Redis est absent, ni la bascule de pilote de stockage.
- L'application mobile n'a aucun test automatisé.

8. Le déploiement

Docker : image `node:22-alpine`, mono-étage. Deux optimisations de cache à savoir expliquer — `package*.json` copié seul avant `npm ci` (la couche d'installation n'est invalidée que si les dépendances changent), et `prisma/` copié avant le reste du code (la génération du client n'est invalidée que quand le schéma change).

Le CLI Prisma est volontairement en `dependencies` et non `devDependencies` : présent au build pour `generate` et à l'exécution pour `migrate deploy`.

⚠ **Deux manques à reconnaître spontanément** : le conteneur tourne en **root** (pas de `USER node`), et l'image n'est pas multi-étages — les outils de compilation restent dedans.

Fly.io : région Paris, deux machines, `auto_stop_machines` et `min_machines_running=0` — l'hébergement coûte zéro euro, la contrepartie est un réveil d'une dizaine de secondes après inactivité. Sonde de santé **applicative** sur `/api/health` : « *une machine qui écoute mais dont la base est injoignable doit être vue comme morte.* »

512 Mo de RAM avec un commentaire honnête dans le fichier : « *256 Mo suffisent au repos, mais pas au client Prisma.* »

Les secrets sont posés par `flyctl secrets set`, jamais dans le fichier versionné.

9. 25 questions probables

Architecture

1. « *Pourquoi Express et pas NestJS ?* » — Besoin d'une API REST classique, seul, sur neuf mois. NestJS apporte une structure que j'aurais dû apprendre en même temps que le métier. La contrepartie : ma structure est une convention, pas une contrainte du cadre.
2. « *Pourquoi REST et pas GraphQL ?* » — Opérations classiques sur des ressources bien identifiées. GraphQL aurait apporté de la souplesse client au prix d'une complexité de cache et d'autorisation que rien ne justifiait.
3. « *Comment garantisiez-vous que le web et le mobile se comportent pareil ?* » — Aucune règle métier dans les clients. Toute règle vit dans l'API. Une règle dupliquée finit par diverger, et une divergence de sécurité ne se voit pas : elle s'exploite.
4. « *Votre API est-elle documentée ?* » — Non, il n'y a pas d'OpenAPI. Les 133 points d'entrée sont lisibles dans les 25 modules de routes. C'est un manque.

Données

5. « *Pourquoi un ORM ?* » — Requêtes paramétrées par construction, migrations versionnées, typage généré depuis le schéma : si je renomme une colonne, le code qui l'utilise ne compile plus. L'erreur remonte à l'écriture.
6. « *Et si l'ORM ne suffit pas ?* » — Trois requêtes en SQL brut, paramétrées, pour un tri par pertinence et un calcul de distance que l'API de requête n'exprime pas.
7. « *Comment gérez-vous les suppressions ?* » — 37 cascades sur les données dérivées, 8 `SetNull` sur les données probantes. Le droit à l'effacement s'applique sans script de nettoyage.
8. « *Vos index sont-ils justifiés ?* » — Treize, posés là où une requête les demande. Un index n'est pas gratuit : il ralentit les écritures. **Et il en manque** — notamment sur la table de liaison coach-client, ma requête la plus fréquente.
9. « *Comment gérez-vous la concurrence ?* » — Par des contraintes d'unicité en base plutôt que par des vérifications applicatives, et par des transactions là où un état partiel serait irrattrapable. Il reste un cas non couvert : le calcul du coach principal.

10. « *Combien de requêtes pour afficher un tableau de bord ?* » — Trop, à un endroit : l'analyse multi-clients fait $1 + 2n$ requêtes. C'est un N+1 identifié, la correction est une requête `in` suivie d'un regroupement en mémoire.

Sécurité

11. « *Comment stockez-vous les mots de passe ?* » — bcrypt, coût 10, sel intégré. Volontairement lent : c'est ce qui rend la force brute coûteuse.
12. « *Que contient votre JWT ?* » — L'identifiant et le rôle. Mais le rôle du jeton ne sert jamais à autoriser : l'utilisateur est rechargé en base à chaque requête.
13. « *Comment révoquez-vous un jeton volé ?* » — **Je ne peux pas**. 30 jours, pas de liste de révocation. La seule option est la rotation du secret, qui déconnecte tout le monde. Le correctif est identifié.
14. « *Un coach peut-il voir les données d'un autre coach ?* » — Non, et c'est testé : deux écosystèmes complets, 18 tentatives d'accès croisé, toutes en 403.
15. « *Quelle différence entre authentification et autorisation ?* » — L'authentification dit qui vous êtes ; l'autorisation, ce que vous pouvez faire **et sur quoi**. Chez moi deux niveaux : le rôle sur la route, la propriété dans le contrôleur.
16. « *Êtes-vous protégé contre l'injection SQL ?* » — Structurellement : Prisma paramètre, et mes trois requêtes brutes utilisent des gabarits balisés qui paramètrent aussi. Aucun appel `Unsafe` dans le projet.
17. « *Le XSS ?* » — L'API ne rend aucun HTML, donc pas de XSS réfléchi. Le XSS stocké est délégué au front — c'est une défense en profondeur qui manque côté serveur.
18. « *Le CSRF ?* » — Sans objet : l'authentification passe par un en-tête, pas par un cookie. Un navigateur n'ajoute jamais cet en-tête sur une requête inter-site.
19. « *Que se passe-t-il si quelqu'un envoie 10 000 requêtes ?* » — 300 par minute et par IP, puis 429. Sur la connexion, 10 échecs par quart d'heure. Les succès ne comptent pas.
20. « *Et les fichiers téléversés ?* » — 5 Mo, extension et type MIME contrôlés, nom remplacé par un UUID. **Mais** le type MIME vient du client et le contenu réel n'est pas inspecté : un fichier déguisé passerait.

Qualité et exploitation

21. « *Comment testez-vous ?* » — Sur une vraie base, en HTTP. Un faux ORM ne teste que mes hypothèses sur l'ORM ; une contrainte d'unicité oubliée ne se voit que sur une vraie base.
22. « *Que se passe-t-il si Redis tombe ?* » — L'application interroge PostgreSQL. Elle ralentit, elle ne s'arrête pas. Un cache indisponible ne doit jamais interrompre un service.
23. « *Comment déployez-vous ?* » — Image Docker sur Fly, migrations d'abord, code ensuite. **En manuel** : il n'y a pas de workflow de déploiement.
24. « *Comment savez-vous que la production va bien ?* » — Une sonde de santé applicative et un redémarrage automatique. **Mais aucune supervision, aucune alerte** : si l'API tombe cette nuit, je l'apprends demain.
25. « *Qu'est-ce que vous referiez autrement ?* » — Trois choses : un schéma de validation par route, la suppression du champ coach hérité, et des tests sur les rendez-vous.
-

10. Les faiblesses

À corriger avant vendredi — court et rentable

⚠️ 1. `sendMessage` ne vérifie aucune propriété. Un utilisateur authentifié peut écrire dans la conversation de deux tiers, et choisir à qui le message est attribué. **C'est la question qui suit naturellement ton explication de l'autorisation à deux niveaux.** Dix lignes et un test.

⚠️ 2. Tes 42 tests unitaires ne tournent pas. `parseSetData.test.js` n'est pas dans la liste `test:raw` : ni `npm test`, ni la CI ne l'exécutent. Le support n'affirme plus qu'ils sont exécutés, donc rien n'est faux — mais si le jury demande « faites-les tourner », le compte tombe de 220 à 126. **Une ligne à ajouter** dans `test:raw`, et le compte est bon.

À assumer, sans les cacher

Faiblesse	La formulation qui tient
Mot de passe à 6 caractères, sans complexité	« L'ANSSI recommande 12. C'est mon premier correctif de sécurité. »
JWT 30 jours non révocable	« Le correctif est identifié : jeton court plus rafraîchissement. »
Validation à la main, couverture inégale	« Le maillon faible, et je le sais. Un schéma par route le réglerait. »
Aucun test sur les rendez-vous	« La fonctionnalité la plus riche en règles, la moins couverte. »
Champ <code>coachId</code> hérité encore lu	« Migration inachevée — c'est ce qui a causé le bug de l'écran d'analyse. »
Pas de pagination sur les conversations	« Croissance non bornée. Le seul endroit qui signale sa troncature est la recherche de salles. »
Conteneur en root, image mono-étage	« Deux corrections d'une ligne que je n'ai pas faites. »
Pas de supervision, retour arrière manuel	« Les deux premières choses que j'ajouterais pour de vrais utilisateurs. »
Migrations appliquées à la main	« Pas de <code>release_command</code> — le déploiement n'est pas complètement automatisé. »
Crons sans verrou distribué	« L'idempotence protège le rappel H-1, pas le J-1. »

Le principe qui doit transparaître : chaque faiblesse est **mesurée, nommée, et son correctif identifié**.
Un jury pardonne une dette connue ; il ne pardonne pas une dette découverte pendant l'entretien.